

Modeling and Verification of Simplified FaB Paxos in TLA+

Ion Tulei

Delft University of Technology
Delft, The Netherlands

Alexandru Verhovetchi

Delft University of Technology
Delft, The Netherlands

Abstract

We build a TLA+ model of Simplified FaB Paxos, the Fast Byzantine Consensus protocol of Martin and Alvisi [1], and we model-check it with TLC. The model covers the common case and the recovery sub-protocol, and it includes the paper’s full threat model: up to f Byzantine agents in each class, acceptors, proposers including the leader, and learners. TLC verifies the three safety properties (CS1, CS2, CS3) exhaustively at $f = 1$, and the two liveness properties (CL1, CL2) under fairness. We add two checks that make the result meaningful: a mutation test shows the model catches real violations, and a load-bearing test shows that the progress certificate is what stops a Byzantine leader. A larger run with two leader changes explored 5.6 million states with no violation but did not finish in our time budget. This is Variant A of the project, where we design the model by hand rather than generate it with GenAI (Variant B), so we wrote the model from the paper’s pseudocode, not from an implementation.

Keywords

TLA+, TLC, model checking, Byzantine fault tolerance, consensus, FaB Paxos

1 Introduction

Consensus lets a set of processes agree on a single value despite failures. In the Byzantine setting, where faulty processes can behave in any way, this is hard and expensive. A process is correct if it follows the protocol; a faulty, or Byzantine, process may deviate in any way, including sending conflicting messages to different peers. So a correct learner is a learner that follows the protocol, and a property stated for correct learners makes no promise about a faulty one. Earlier Byzantine consensus protocols need three communication steps in the common case, where the non-Byzantine case needs two [1]. PBFT [5], used for state-machine replication, takes four steps per request. FaB Paxos [1], the protocol we study, reaches agreement in two steps in the common case, which matches the lower bound for two-step Byzantine consensus. The price is more acceptors: FaB needs $5f + 1$ of them to tolerate f Byzantine faults, and the paper proves this bound is tight [1].

The reason for modeling the protocol in TLA+ [3] is that a Byzantine consensus protocol can run its steps in a huge number of orders, and only a few of those orders break correctness. Hand proofs are easy to get wrong, and a test run only covers the orders it happens to execute, so it can miss the rare bad one. A TLA+ model checked with TLC explores every reachable state of a bounded instance, so it finds corner cases a reader would miss. FaB’s recovery sub-protocol, where a new leader must not undo an earlier decision, is the kind of subtle logic that benefits from this check.

The protocol’s participants are processes, and each acts as one or more agents: a proposer (which proposes values), an acceptor (which helps choose a value), or a learner (which learns the chosen value). Consensus is defined by three safety properties and two liveness properties [1]:

- CS1: only a value that has been proposed may be chosen.
- CS2: only a single value may be chosen.
- CS3: only a chosen value may be learned by a correct learner.
- CL1: some proposed value is eventually chosen.
- CL2: once a value is chosen, every correct learner eventually learns it.

Our objective is a faithful TLA+ model of a simplified subset of FaB Paxos and a machine-checked argument that it satisfies these properties. We chose FaB from the Variant A options (Chord, PBFT, FaB) because it is a compact protocol with a clear two-step common case and an interesting recovery mechanism, the progress certificate, which defends against a faulty leader. The paper gives precise pseudocode (Figures 1 and 2) [1], which suits modeling by hand.

We set three research questions:

- RQ1: can we build a faithful TLA+ model of FaB’s common case and recovery under the paper’s full per-class Byzantine threat, with one Byzantine acceptor, proposer, and learner?
- RQ2: do the safety properties (CS1-CS3) and the liveness properties (CL1-CL2) hold under bounded model checking, and do the checks actually catch faults?
- RQ3: how does the model-checking cost scale across configurations, and what keeps the model small enough for TLC to finish?

Our method is to write the spec by hand from Figures 1 and 2 of [1], check it with TLC, validate the checks with a mutation test and a load-bearing test, study how the cost scales across configurations, and score the model against the SysMoBench [8] metrics. In short: the model is faithful, and the safety and liveness properties hold at $f = 1$ with one leader change, across 147,340 and 4,642,112 states; the mutation and load-bearing tests confirm the checks catch real faults; the value domain and symmetry reduction have the biggest effect on the model-checking cost; and the work surfaced a real bug in how we defined when a value is chosen, which we found and fixed.

This report covers our modeling choices (Section 2), the correctness specifications and the verification results (Section 3), the limitations (Section 4), a comparison to related work (Section 5), our use of GenAI tools (Section 6), and references.

2 Formal Modeling

We wrote one protocol module, `FaBPaxos.tla`, and three small model-checking wrappers that reuse it: `MCFaBPaxos(safety)`, `MCFaBPaxos`

Liveness (liveness), and MCFaBPaxosTrace (example traces). There is no duplicated protocol logic.

In the common case [1], the leader sends a PROPOSE with a value to all acceptors. Each correct acceptor accepts the value and sends an ACCEPTED to the learners. A learner decides once it sees a fast quorum of matching ACCEPTED messages, a group large enough that the decision stays safe even when up to f of the votes come from faulty acceptors, five at $f = 1$. Learners share a LEARNED message so a lagging learner can catch up. That is the two-step path: PROPOSE, then ACCEPTED, and the learner decides.

A concrete run makes this clear, the shortest one TLC found, saved in traces/common-case. The leader p1 proposes v_0 at proposal number 0. Acceptors a1 to a5 each accept and send ACCEPTED ($v_0, 0$). Five matching ACCEPTED messages now sit in the buffer. That is the fast-learn quorum at $f = 1$, so learner l1 decides ($v_0, 0$). The whole run is eight states: one PROPOSE, five ACCEPTED, and one decision.

Recovery starts when proposers stop seeing progress and suspect the leader. They cannot tell whether it is slow or faulty, so they replace it: a new leader takes over at a higher proposal number. It sends a QUERY, each correct acceptor REPLIES with the value it last accepted, and the new leader collects these signed replies into a progress certificate. The certificate shows which value, if any, may already be chosen, so the new leader proposes a value the certificate vouches for. This is what stops a Byzantine leader's poisonous write, proposing different values to different acceptors to split the decision.

The progress certificate is the set of $a - f$ signed REP replies, where a is the number of acceptors, so five at $f = 1$. Following [1], it vouches for a pair (v, pn) , a value v at proposal number pn , when no other value appears $\lceil (a - f + 1)/2 \rceil = 3$ or more times among the replies. If one value holds a majority of the replies, the certificate vouches for that value alone; if none does, it vouches for any value, so the new leader may pick freely.

2.1 State and messages

We follow the style of Lamport's Paxos. t1a [4]: a single global set of messages plus per-agent state. The state variables are the message buffer `msgs`, the per-acceptor `accAccepted` and `accLargestPN`, the per-learner `lnLearned`, the per-proposer `prSuspects`, the current leader, and the current proposal number `currentPN`. The buffer `msgs` is append-only and is a set, so the model gets message re-ordering and duplication for free, and a dropped message is just an action that does not fire.

Five message types carry the protocol: PROPOSE, ACCEPTED, LEARNED, QUERY, and REP. Each message that comes from an agent carries that agent's identity (for example, an ACCEPTED carries the acceptor that sent it). This identity field is how we model the paper's cryptographic soundness assumption [1]: a Byzantine agent can stamp only its own identity, so it cannot forge a message from a correct agent.

2.2 Actions

Each action maps to a handler in Figure 1 (common case) or Figure 2 (recovery) of [1]:

- **LeaderPropose**: a correct leader proposes one value per proposal number. At the first term the choice is free; at later terms the value must be vouched for by a progress certificate.
- **Accept**: a correct acceptor accepts a proposed value under the Figure 2 guards (not stale; no downgrade, it will not accept a lower proposal number once it has replied at a higher one; and a value switch only if a certificate vouches for the new pair).
- **Learn**: a correct learner decides once a fast quorum of matching ACCEPTED messages exists.
- **LearnViaPull**: a correct learner adopts a value on $f + 1$ matching LEARNED reports from peers.
- **SuspectLeader, LeaderChange**: the abstracted leader election. When enough proposers suspect the leader, a new leader takes over at the next proposal number.
- **Query, Reply**: the new leader gathers a progress certificate from the acceptors during recovery.
- **ByzantinePropose**: a Byzantine leader proposes any value with no limit, which models the poisonous write.

At $f = 1$ the model has six acceptors (one Byzantine), four proposers (one Byzantine, which can become leader), and four learners (one Byzantine). The paper bounds faults per class, so each class holds up to f Byzantine agents, and a single process may act as more than one role at once, for example as both an acceptor and a learner [1]. We model the three classes as separate sets, so $f = 1$ gives one faulty agent in each class: one acceptor, one proposer, and one learner. The quorum thresholds are the paper's [1]: a fast-learn quorum of $\lceil (a + 3f + 1)/2 \rceil = 5$, a chosen threshold of $\lceil (a + f + 1)/2 \rceil = 4$, and a pull cross-check of $f + 1 = 2$.

The first two thresholds connect in CS3 [1]. A learner decides on five matching ACCEPTED messages. At $f = 1$, at most one comes from a faulty acceptor, so at least four come from correct ones, which is the chosen threshold. A value a correct learner decides is therefore always chosen.

2.3 Key design decisions

The progress certificate is a guard, not message data. The paper's leader attaches the certificate to its PROPOSE. We do not store the certificate in the message. The certificate is a set of signed REP messages, and Lemma 4's proof [1] depends on the replies being signed, not on the leader bundling them, so we let both the leader's value choice and the acceptor's value-switch check read the signed REP messages that already sit in `msgs`. This is faithful, and it avoids storing a set inside a message, which would blow up the state space (Section 3.5).

Chosen is monotonic over the message log. A pair is chosen when enough correct acceptors have accepted it. We count the ACCEPTED messages from correct acceptors in `msgs`, not the acceptors' current state. Because `msgs` is append-only, this count never decreases, so the relation is monotonic: once a pair is chosen it stays chosen. This matches the paper's past-tense definition [1] ("have accepted") and is needed for CS3 to hold under recovery.

Byzantine acceptor and learner messages are injected at the start. A Byzantine agent may send anything, so we model it as having already sent everything it could. This is the maximal

adversary, so it is sound for safety, and it removes the branching that those actions would otherwise add.

Abstractions. We model single-decree consensus (one value, not a sequence), we abstract leader election as a nondeterministic action, we drop the SATISFIED message (it drives only the leader’s retransmission stop condition, a liveness detail), and we treat signatures as identity stamps rather than real cryptography. Section 4 lists these as limitations.

Because this is Variant A, the model comes only from the paper’s pseudocode [1]. We did not use or look at any existing FaB implementation.

3 Correctness Specifications and Verification Results

3.1 Correctness properties

We encode CS1, CS2, and CS3 as state invariants and CL1 and CL2 as temporal properties. Chosen(v , pn) and Vouches(...) are derived operators that the invariants build on. For example, a pair is chosen when enough correct acceptors have accepted it, and CS2 says any two chosen pairs carry the same value:

```
Chosen(v, pn) ==
  NumAcceptedMessagesCorrect(v, pn) >= ChosenThreshold

CS2Inv ==
  \A v1, v2 \in Value, pn1, pn2 \in PN :
    Chosen(v1, pn1) /\ Chosen(v2, pn2) => v1 = v2
```

Here Value is the set of candidate values and PN the set of proposal numbers, 0 to MaxPN. CS3Inv quantifies over the correct learners only, since a Byzantine learner never records a real decision. The safety run uses SPECIFICATION Spec; the liveness run uses SPECIFICATION LiveSpec, which adds weak fairness on the correct actions.

3.2 Setup

Safety runs at $f = 1$ with the full per-class Byzantine threat (one Byzantine acceptor, one proposer, and one learner at once), two candidate values, and one leader change (MaxPN = 1). It uses symmetry reduction over the correct agents and a state constraint that bounds the message buffer, both sound for invariant checking. Liveness runs under the paper’s good-period assumption (a correct stable leader, set by making the proposer set fault-free), with a single value, and with symmetry and the state constraint turned off, since both are unsound for liveness.

3.3 A modeling bug we found and fixed

While we were adding the recovery sub-protocol, TLC reported CS3Inv is violated on a 14-step trace. The trace runs like this: the first leader proposes v_0 at proposal number 0; four correct acceptors and the Byzantine acceptor send ACCEPTED for $(v_0, 0)$; the proposers suspect the leader, and a leader change moves to proposal number 1; one correct acceptor re-accepts $(v_0, 1)$, which Figure 2 of [1] allows because the value is the same and the number is higher; a learner then sees five matching ACCEPTED for $(v_0, 0)$ and learns it, and the CS3 check returns false.

The bug was in our model, not the protocol. Our first Chosen(v , pn) counted acceptors whose current accAccepted equals (v, pn) .

That is a snapshot. The paper [1] defines a chosen pair as one that four correct acceptors have accepted, the chosen threshold at $f = 1$, which is past tense and never reverts. Once the acceptor moved from $(v_0, 0)$ to $(v_0, 1)$, the snapshot dropped $(v_0, 0)$ below the threshold, even though the ACCEPTED message still sat in the log. We changed Chosen to count ACCEPTED messages from correct acceptors in the append-only msgs (Section 2), which is monotonic and matches the paper.

The fix shows up in the model checking (Table 1). With the snapshot definition TLC reports the CS3 violation above; with the monotonic definition the same recovery scope at MaxPN = 1 is clean. The counterexample turns into a non-violation, since $(v_0, 0)$ stays chosen.

Table 1: The Chosen relation, buggy versus fixed.

Chosen definition	Result at MaxPN = 1
snapshot of accAccepted (buggy)	CS3 violated, 14-step trace
count over msgs (fixed)	clean, 147,340 states

3.4 Verification results

Table 2 reads top to bottom as the main result followed by the checks that back it up. The first row is the headline: at MaxPN = 1 with one Byzantine agent in each class, TLC checked TypeOK (the type-correctness invariant), CS1, CS2, and CS3 across all 147,340 reachable states and found no error, so safety holds for the whole bounded space, not a sample. The second row is liveness: CL1 and CL2 hold across 4,642,112 states under fairness and a correct stable leader. That count is larger because the liveness run drops symmetry, which is unsound for liveness (Section 3.5). The next two rows are the checks on the checks, and the last row pushes the scope to two leader changes. We explain all three below.

The checks can fail (mutation test). A check that can never fail catches nothing, so a clean run on its own would mean little. We tested that the checks can fail: we weakened the fast-learn quorum and re-ran. TLC found a CS3 violation in under a second, which shows the model detects real safety failures. We reverted the change.

The certificate is load-bearing. We removed the acceptor’s certificate check and re-ran with the Byzantine leader active. TLC found a CS2 violation in 14 seconds. The counterexample is the paper’s poisonous write: a value is chosen at the first term, the Byzantine proposer becomes leader, proposes a different value at the next term, and correct acceptors accept it because the defense is gone, so a second value is chosen. This shows the certificate check is what enforces safety, not an accident of the model.

MaxPN = 2. We raised the bound to two leader changes to exercise the cross-recovery argument. The run explored 5.6 million states with no violation but did not finish in five hours; the three-term state space is larger than explicit-state TLC handles on a laptop. This is a tooling limit, not a defect: the growth was steady and no violation appeared.

Table 2: TLC verification results. The first two rows are the main safety and liveness runs; the rest are the checks that back them up and a larger scope that did not finish.

Run	Distinct states	Depth	Runtime	Result
Safety, MaxPN = 1, full Byzantine	147,340	31	10m 25s	clean, no error
Liveness, CL1 and CL2	4,642,112	30	16m	clean, both hold
Mutation test (weakened quorum)	n/a	shallow	<1s	CS3 violation, expected
Load-bearing test (no certificate check)	n/a	15-step	14s	CS2 violation, expected
Safety, MaxPN = 2, full Byzantine	5,604,047 explored	31	killed at 5h	did not finish

3.5 Performance study

We varied one parameter at a time from the safety baseline, holding $f = 1$ and MaxPN = 1 fixed, and measured cost (Table 3). We fix those two because $f = 2$ needs $5f + 1 = 11$ acceptors, out of reach for explicit-state checking, and MaxPN = 1 already drives one full leader change. The study varies the parameters we can change while the model still finishes: value-domain size, Byzantine presence per class, symmetry reduction, and worker count. Full logs and the run table live in spec/perf/.

Three findings stand out. First, the value domain is the biggest factor in the state count: cutting it from two values to one shrinks the count about eightfold, from 147,340 to 18,316, since each acceptor can accept any proposed value at each term. Second, symmetry reduction is essential. Turning it off does not finish: it reached 5.4 million states in under four minutes while still at depth 20, against 147,340 states at depth 31 with symmetry on, a quotient of at least 37x, and unbounded since the run never converged. Third, Byzantine faults are not the main cost, and removing them does not shrink the model. Full Byzantine faults cost more than an acceptor-only adversary, 147,340 against 32,288, because correct agents react to adversarial PROPOSE and LEARNED messages. Dropping every Byzantine process does not finish, though: a Byzantine process is a fixed injected baseline, while a correct process branches through its actions, so an all-correct model has wider per-state fanout. Extrapolating the single-value no-Byzantine run (15,024) by the eightfold value factor puts the full no-Byzantine model near 120,000 states, the same order as the baseline. Worker scaling is sub-linear, near 2.4x at four workers against one on the small configuration (a single value and no Byzantine agents), since the workers share one fingerprint set and one state queue.

Symmetry reduction is worth a word, since it is the largest single factor and we turn it off for liveness. The correct agents in a class are interchangeable, so two states that differ only by renaming them have the same shape. For example, a state where acceptors a2 and a3 have accepted v_0 matches the state where a3 and a2 have, and TLC keeps one per group and skips the rest, which cuts the count. This is sound for invariants, which hold or fail no matter the names. It is not sound for liveness: a liveness counterexample is often an unfair run where one chosen agent is starved forever, and merging the runs that differ only in which agent is starved can hide that one. So the liveness check keeps every agent distinct and drops symmetry.

The state constraint is the other reduction. It tells TLC to stop exploring once the message buffer passes a set size, which keeps the search finite. It is sound for invariant checking, since every state

TLC reaches is a real reachable state, so the invariants are genuinely checked; the only cost is that a violation beyond the bound could be missed. For liveness it is unsound, because cutting a run short can hide an infinite behavior that breaks a property, so the liveness run turns it off too.

State counts and search depths are exact and reproducible. Run-times are indicative, measured on one laptop under sustained load where later runs may run on a throttled CPU, so we read them as orders of magnitude.

3.6 SysMoBench evaluation

SysMoBench [8] scores a TLA+ model on four metrics: syntax, runtime, conformance, and invariant correctness. It is built to evaluate AI-generated models over its own eleven systems (etcd Raft, ZooKeeper, and others), which fits more Variant B of the project. We wrote our model by hand and FaB is not one of its systems, so we cannot run its harness. We instead apply its four metrics as a rubric, using the same tools it wraps (SANY and TLC), and we report a verdict per metric rather than a single weighted score (Table 4).

Syntax. SANY parses all four modules with no error. The next-state relation is a disjunction of nine named actions, so the action-decomposition check is met.

Runtime. TLC explores the full safety state space (Section 3.4) with no runtime error. A coverage run records how many times each action fired: all nine fire, from Accept at 13,214 distinct states down to the recovery actions LeaderChange and Query at 51 each, low because MaxPN = 1 allows a single leader change. No action has a zero count, so the spec has no dead transitions. The full per-action table is in spec/eval/.

Conformance. This metric needs a running implementation and execution traces captured from it, which Variant A does not have: we modeled the protocol from the paper, not from code. Our spec/traces/ files are generated by TLC from the spec itself, so using them here would be circular and would not test conformance to an external system. We report this metric as not applicable, a structural property of Variant A rather than a gap in the model.

Even with an implementation, we would face several challenges in evaluating conformance. FaB has no canonical implementation; a third-party DistAlgo version exists [13], but it does not match our simplified subset, so sourcing matching code is the first barrier. SysMoBench reads execution traces in a fixed format, so we would instrument the code to log each relevant event, a message sent, a value accepted, a value learned, a leader change, and map those fine-grained events onto our abstract actions. The harder problem is the gap between the code and our model. We model a simplified

Table 3: Controlled performance runs. One parameter changes per run from the safety baseline ($f = 1$, $\text{MaxPN} = 1$).

Run	Parameter from baseline	Distinct states	Depth	Runtime	Outcome
baseline	reference	147,340	31	10m33s	completed
one value	two values to one	18,316	30	1m03s	completed
acceptor-only Byzantine	no Byzantine proposer or learner	32,288	31	19m47s	completed
no Byzantine	no Byzantine process	17,610	25	30m (cap)	did not finish
symmetry off	symmetry reduction disabled	5,423,956	20	3m39s	did not finish
one worker	small config, single worker	5,547	23	35m (cut)	did not finish
four workers	small config, four workers	15,024	34	40m51s	completed

Table 4: SysMoBench metrics applied as a rubric to our hand-written model.

Metric	What it checks	How we evaluate it	Result
Syntax	The spec parses, and the next-state relation is decomposed into named actions	SANY parses each module; inspect Next	Pass
Runtime	TLC model-checks the spec with no runtime error, and every action is reachable	Run TLC, then run TLC with <code>-coverage</code> for per-action counts	Pass
Conformance	Each spec transition matches a running implementation, checked by validating captured execution traces [11]	Replay implementation traces against the spec	Not applicable
Invariant	The spec satisfies the expert-written properties, non-trivially	TLC checks the invariants and temporal properties; mutation tests confirm the checks can fail	Pass

subset: we drop the SATISFIED message, we abstract leader election, we treat signatures as identity stamps, and we read the progress certificate from the buffer instead of carrying it in the PROPOSE. A real run shows behavior the spec leaves out, so we would either enrich the spec to match the code or project each trace down to the modeled part, and either choice can hide a true divergence if the mapping is wrong. A correct-only trace tests just the common case, so capturing Byzantine runs would also need a fault-injection harness.

Invariant. TLC verifies TypeOK, CS1, CS2, and CS3 across all 147,340 safety states, and CL1 and CL2 across 4,642,112 liveness states (Section 3.4). The mutation, load-bearing, and liveness-fairness tests show the properties can fail when the protocol is broken. This is the metric our work targets most directly.

We satisfy three of the four metrics. Conformance is the one we cannot compute, for the structural reason above, not for a defect in the model.

4 Limitations

We are honest about what the model does not cover:

- **Fault budget $f = 1$.** We verify at $f = 1$. $f = 2$ needs eleven acceptors and the state space is too large for TLC, so higher f is out of reach with explicit-state model checking.
- **Models the original $5f + 1$ bound.** A later result [9] tightened the bound for two-step Byzantine consensus to $5f - 1$, so our $f = 1$ instance uses two more acceptors than an optimal protocol needs. We model the 2006 protocol on purpose, and we note the gap (Section 5).

- **MaxPN = 2 not finished.** The two-leader-change run did not complete in five hours. The clean exhaustive result is at $\text{MaxPN} = 1$ (one leader change).
- **Single-value liveness model.** Liveness is about progress, which does not depend on the number of candidate values, so the liveness run uses one value to stay tractable. Safety uses two values, which CS2 needs.
- **No implementation, so no conformance.** We modeled the protocol from the paper, so the SysMoBench conformance metric does not apply (Section 3.6).
- **Abstracted leader election.** We model election as a non-deterministic action with a strictly higher proposal number, as the paper itself does [1].
- **Dropped SATISFIED message.** It drives only the leader’s retransmission stop condition and does not affect the safety properties.
- **Liveness good-period assumption.** Liveness is impossible under full asynchrony (FLP) [7], so we check it under a correct stable leader, as the paper does [1].

5 Discussion of Related Work

We compare our work with related efforts on a few clear axes: the fault model, the number of communication steps in the common case, the number of processes at $f = 1$, what each one models and verifies, and whether the analysis is static or dynamic. Table 5 sets the protocol numbers side by side.

Modeling: Lamport’s Paxos and Byzantine Paxos in TLA+. This is the closest match in method. Lamport’s Paxos.tla [4] picks the proposed value inside the Phase2a guard and stores only the (value, ballot) pair in the message, not the votes that justify it [2]. We use the same idiom: VouchedAtPN reads the signed REP messages

Table 5: How FaB compares with related consensus protocols. The step counts are common-case agreement steps; PBFT’s full state-machine request is four.

Protocol	Faults	Common-case steps	Processes at $f = 1$	Faulty-leader defense
Paxos	crash	2	$2f + 1 = 3$	not needed
PBFT	Byzantine	3	$3f + 1 = 4$	view change
FaB (this work)	Byzantine	2	$5f + 1 = 6$	progress certificate
FaB, tight bound (Kuznetsov 2021)	Byzantine	2	$5f - 1 = 4$	progress certificate

already in the buffer, and the PROPOSE carries only (value, pn). That one choice is what keeps our recovery scope tractable, as the performance numbers show (Section 3.5). Our model is stronger in three ways: it carries the full per-class Byzantine threat at once (a lying acceptor, a lying leader, and a lying learner), it checks liveness (CL1, CL2), and it adds the mutation and load-bearing tests that confirm the checks can fail, where many teaching specs check safety alone. It is weaker in scope: Lamport’s specs are general and hold for any f , and his Byzantine Paxos [10] is a fuller treatment of the faulty-leader case, while ours is a fixed $f = 1$ instance of a simplified subset. It does fill a small gap, since FaB had no canonical public TLA+ spec for us to start from.

Verification and protocols: PBFT and FastPaxos. PBFT [5] is the Byzantine baseline FaB sets out to beat. It uses $3f + 1$ replicas, and its agreement takes three communication steps; a full state-machine request takes four [1]. FaB uses $5f + 1$ acceptors and reaches consensus in two steps. The trade is real and goes both ways. FaB removes one step, which lowers latency and helps under high load or geo-distribution, but at $f = 1$ it runs 6 acceptors against PBFT’s 4, a 50% jump in nodes, and it sends more messages per decision. PBFT is also deployed and battle-tested, while FaB stays mostly in the theory and teaching world. So FaB’s strength is latency, and its weakness is cost and maturity. FastPaxos [6] is the crash-fault protocol that also decides in two steps. The contrast is sharp: it needs no progress certificate, because a crashed acceptor goes silent and cannot lie, while FaB needs the certificate to stop a Byzantine leader. Our load-bearing test (Section 3.4) is a concrete picture of that cost: drop the certificate check and a Byzantine leader breaks CS2 with a poisonous write, an attack the crash model has no analog for. We do not compare raw TLC state counts with these protocols, since the protocols and the way each spec is written differ too much for the numbers to mean the same thing.

Resilience: Revisiting Optimal Resilience of Fast Byzantine Consensus (Kuznetsov, Tonkikh, and Zhang, 2021). This [9] is the honest weak point of the algorithm we chose. FaB [1] claims $5f + 1$ is the tight lower bound for two-step Byzantine consensus. Kuznetsov and co-authors show the real tight bound is $5f - 1$, so the $5f + 1$ figure was not minimal. At $f = 1$ the optimal protocol uses 4 processes, the same count as PBFT, while keeping the two-step latency. Our model verifies the original 2006 version, so our instance runs two acceptors heavier than an optimal protocol needs (Section 4). We model the 2006 paper on purpose, and we state the gap rather than hide it.

Testing and trace validation: ByzzFuzz and SEFM trace validation. These show what our static check does not. We explore every reachable state of a bounded instance, so we catch design bugs with no false negatives inside the bound; that is how we found

our own CS3 bug (Section 3.3). ByzzFuzz [12] takes the dynamic route: it injects random network and process faults into a running BFT system and watches for safety breaks, and it found a real bug in the Ripple (XRP Ledger) implementation, a class of bug we cannot reach because we have no code. Trace validation [11] sits between the two: it records a real run and checks each step against the spec. The honest limit is the one the conformance metric already exposed: our model is correct against the paper and against itself, not against any implementation (Section 3.6).

Our work sits in a clear spot. The contribution is a hand-written TLA+ model of FaB, a protocol without a canonical public spec, that carries the full per-class Byzantine threat and checks both safety and liveness, backed by tests that confirm the checks can fail. The limits are clear too: we verify at $f = 1$, we model a simplified subset, we model the superseded $5f + 1$ bound, and we have no implementation to validate against. On performance, head-to-head TLC counts and runtimes for these other protocols are rarely published, and the models differ, so a direct table would mislead. That comparison stays qualitative, on the fault model, the steps, the node count, and what is tractable, and the only hard numbers are our own (Sections 3.4 and 3.5).

6 Statement of GenAI Use

We used AI assistants in a limited way during the project. We used them to correct grammar and spelling in our reports and in the README. We also used ChatGPT to explain theoretical topics and to help debug problems when we were stuck.

References

- [1] J.-P. Martin and L. Alvisi. Fast Byzantine consensus. *IEEE Transactions on Dependable and Secure Computing*, 3(3):202–215, 2006.
- [2] L. Lamport. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.
- [3] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [4] L. Lamport. Paxos.tla, TLA+ Examples repository. <https://github.com/tlaplus/Examples>, accessed June 2026.
- [5] M. Castro and B. Liskov. Practical Byzantine fault tolerance. In *Proceedings of OSDI*, 1999.
- [6] R. Boichat, P. Dutta, S. Frolund, and R. Guerraoui. Reconstructing Paxos. *ACM SIGACT News*, 34(2), 2003.
- [7] M. Fischer, N. Lynch, and M. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.
- [8] SysMoBench: Evaluating AI on formally modeling complex real-world systems. arXiv preprint arXiv:2509.23130, 2025.
- [9] P. Kuznetsov, A. Tonkikh, and Y. X. Zhang. Revisiting optimal resilience of fast Byzantine consensus. In *Proceedings of PODC*, 2021. Extended version arXiv:2102.12825.
- [10] L. Lamport. Byzantine Paxos (Dr. TLA+ series), TLA+ DrTLAPlus repository. <https://github.com/tlaplus/DrTLAPlus>, accessed June 2026.
- [11] Validating traces of distributed programs against TLA+ specifications. In *Proceedings of SEFM*, 2024.

- [12] L. N. Winter, F. Buse, D. de Graaf, K. von Gleissenthall, and B. Kulahcioglu Ozkan. Randomized testing of Byzantine fault tolerant algorithms. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):757–788, 2023.
- [13] FaB-DistAlgo: a DistAlgo implementation of Fast Byzantine Paxos. GitHub repository, <https://github.com/lakshmanvinu/FaB-DistAlgo>, accessed June 2026.